

Department of Mechanical Engineering

Master of Science in Robotics and Automation

Capstone Design Project Report – Summer 2025

Developing a Multi-Task Robot Work-Cell

Dakshitha (Dak) Amarasinghe

Supervised by

Dr. Michael Neumann, Dr. Christopher Kitts, and Dr. Manoj Sharma

Abstract

In the present, robotic work cells are very essential within industrial environments, particularly in sectors such as marine, aerospace, automotive, manufacturing and agriculture. They represent a significant advancement in automation, aiming to perform tasks that are highly complicated and time-consuming for human workers to handle effectively. Robotic work cells offer numerous advantages, including increased accuracy and precision, high productivity, and improved operational efficiency. Robotic work cells typically consist of robotic arms, sensors, controllers, conveyors, and other specialized tools, all integrated within a coordinated setup to streamline workflows and minimize errors. Additionally, they are capable of operating in hazardous environments where human safety would be at risk, further highlighting their importance. As industries continue to evolve, the integration of robotic work cells stands as a critical factor in enhancing performance, safety, and technological growth. Hence, the role of robotic work cells is becoming increasingly vital in shaping the future of industrial processes.

Acknowledgement

I would like to express my special thanks to my advising Drs Christopher Kitts, Michael Neumann, and Manoj Sharma for their continuous guidance, support and feedback on this project. I also wish to thank my team members Atharv Khopkar and Jixing Shi for their cooperation and dedicated teamwork throughout this project. Finally, I would like to offer my gratitude to the faculty and my colleagues of the department of mechanical engineering for providing supportive academic environment by giving access to the robotics systems/maker labs and necessary resources to complete this progress.

Table of Contents

Abstract 2

Acknowledgement 3

1 Introduction 6

1.1 Applications for Robot work-cells in specific world-wide industries 6

1.2 Why multi-task robot work-cells are more important 7

1.3 Project Mission Statement 8

2 System overview 9

2.1 Work-cell Description 9

2.2 Coordinate frames of the work-cell 10

2.3 Concept of Operations 12

2.4 Mechanical Diagram 14

2.5 Component Block Diagram 16

2.6 High Level Software Flowchart 17

3 Subsystems 19

3.1 Vision System 19

3.1.1 YOLOv8 for Custom Object Detection 19

3.1.2 Integration of the Zed camera with Yolov8 Model 20

3.1.3 Vision Sub-Systems and Their Results 21

3.1.4 Tool Detection and Position Tracking 21

3.1.5 Color Detection 22

3.1.6 Size Calibration and Identifying the Size Category 24

3.1.7 Rotation angles of the tool and arms end-effector calibration 27

3.1.8 Velocity Tracking 28

3.2 Rotrics DexArm (Robot Arm) 28

3.2.1 Overview 28

3.2.2 Coordinate Transformation ……. 29

3.2.3 The workspace of the DexArm …… 31

3.2.4 Controlling the DexArm using Python API (G-Code) for the Work-Cell …… 32

4 Final Test Program ……. 34

5 Real-time Data Transmission ……. 38

6 Final Data Acquisition to Calibrate the Accuracy of The Work-Cell …… 46

6.1 Experiment Overview …….. 46

6.2 Test Descriptions and Their Results ……... 46

6.3 Evaluation of Test Results ……... 49

7 Conclusion ……... 50

References ……... 52

Chapter 1

Introduction

1.1 Applications for Robot work-cells in specific world-wide industries

Industrial robot work cells have become an essential part of modern manufacturing, driving substantial improvements across various aspects of production. These robotic systems are specifically designed to enhance efficiency, safety, and product quality, while simultaneously reducing costs. By operating at high speeds and working tirelessly without the need for breaks, industrial robots streamline workflows and eliminate bottlenecks, boosting productivity and throughput in ways human workers cannot match. Additionally, robots can take on high-risk tasks in hazardous environments, reducing the health and safety risks that come with manual labor. Their precision and consistency also contribute to better product quality, as robots can execute tasks with exacting accuracy, reducing the chance of human error and ensuring uniformity in production. Furthermore, integrating robots into the production process brings significant cost-saving benefits. Robots not only improve operational efficiency, but also help to minimize waste, reduce workplace injuries, and lower the potential for costly accidents. The growing trend of automation in the manufacturing sector highlights these advantages, with many industries embracing robotics as a means to enhance both productivity and profitability.



Figure 1.1: Applications of robot work-cells (Used without permission [1])

Figure 1.1 illustrates a robot work-cell application. Robots have found applications across a broad spectrum of industrial tasks, such as assembly, material handling, welding, picking, packing, palletizing, and quality control. In each of these roles, robots provide unparalleled advantages, from eliminating bottlenecks and speeding up processes to increasing the precision of inspections and enhancing safety in challenging environments. With continued advancements in robotics, the role of automated work cells in reshaping the manufacturing landscape is set to expand, driving even greater levels of performance and innovation in industries worldwide.

1.2 Why multi-tasked robot work-cells are more important

Multitasking robots are rapidly transforming the landscape of modern manufacturing, offering significant advantages in efficiency, flexibility, and cost-effectiveness. As industries face increasing demands for adaptability and precision, these versatile machines provide a powerful solution to meet the challenges of today's dynamic production environments. From high-mix, low-volume manufacturing to handling repetitive, mundane tasks, multitasking robots allow manufacturers to stay competitive in a fast-paced market. They excel in a variety of tasks, including assembly, packaging, quality checks, and more, all while being easily reprogrammed to accommodate fluctuating workloads and changing production requirements. With their ability to seamlessly shift between tasks without extensive reconfiguration, multitasking robots not only enhance operational efficiency but also improve resource utilization. Moreover, the integration of offline robot programming and real-time data insights further amplifies their value, empowering manufacturers to make data-driven decisions and reduce downtime. As businesses continue to optimize their processes and scale production, multitasking robots will play an increasingly pivotal role in shaping the future of manufacturing automation. The following figure 1.2 shows a work-cell with multiple robots.



Figure 1.2: Multi robot work-cell (Used without permission [2])

1.3 Project Mission Statement

In modern industrial automation, the demand for flexible, precise, and scalable robotic systems has significantly increased due to evolving manufacturing requirements and high product variability. Traditional robotic work cells are often designed for a single, repetitive task, limiting their adaptability, accuracy, and efficiency in both static and dynamic production environments. These limitations result in increased operational costs, longer setup times, and reduced overall productivity. The primary objective of this project is to design and develop a high-performance, multi-functional robotic work-cell capable of executing multiple tasks within a unified system. The proposed solution must ensure high accuracy, precision, and operational efficiency while remaining robust in both static and dynamic conditions. The system must integrate real-time sensing, advanced object recognition, and precise robotic actuation to support single and multiple object detection scenarios effectively. To ensure consistent performance, the work-cell will undergo repeatable testing protocols in various sub systems to identify and address sources of error, enhance system reliability, and optimize task execution. This iterative testing approach is essential for refining the system's decision-making processes and mechanical operations in real-time.

Chapter 2

System Overview

2.1 Work-cell Description

This project focuses on the development and implementation of an integrated robotic work cell designed to perform practical tasks such as packing, assembling/disassembling, and sorting with enhanced performance and precision. The core objective is to create a high-efficiency system capable of operating in real time by integrating three primary hardware components: a Zed stereo camera as a vision sensor, a conveyor belt for object movement, and the ‘RotricsDexarm’ (robot arm) for object manipulation. These subsystems are connected into a unified structure, allowing seamless data flow and coordinated operation within the work cell. The conveyor belt transports various tools at variable speeds, while the vision system continuously monitors and analyzes the objects in motion or stationery. Utilizing a custom-trained YOLOv8 computer vision model, the system can accurately detect and classify tools specifically hammers, wrenches, and screwdrivers. It also tracks each object’s position and velocity in real time within the camera frame. The software architecture supporting this system is composed of multiple specialized sub-programs, each responsible for tasks such as tool category detection, color identification, object position/velocity tracking, size calibration, and robot arm control. When a tool is detected, the conveyor halts at a precise location to allow the robot arm to execute pick-and-place operations with greater performance. accuracy and efficiency. The system is robust enough to handle both single and multiple object detection scenarios, enabling it to function reliably in both static and dynamic environments. Through the integration of real-time sensing, information of the object recognition (Ex: category, size, color), and precise robotic actuation, this project demonstrates a

scalable and high-performance robotic work cell suitable for a range of industrial automation tasks.

2.2 Coordinate frames of the work-cell

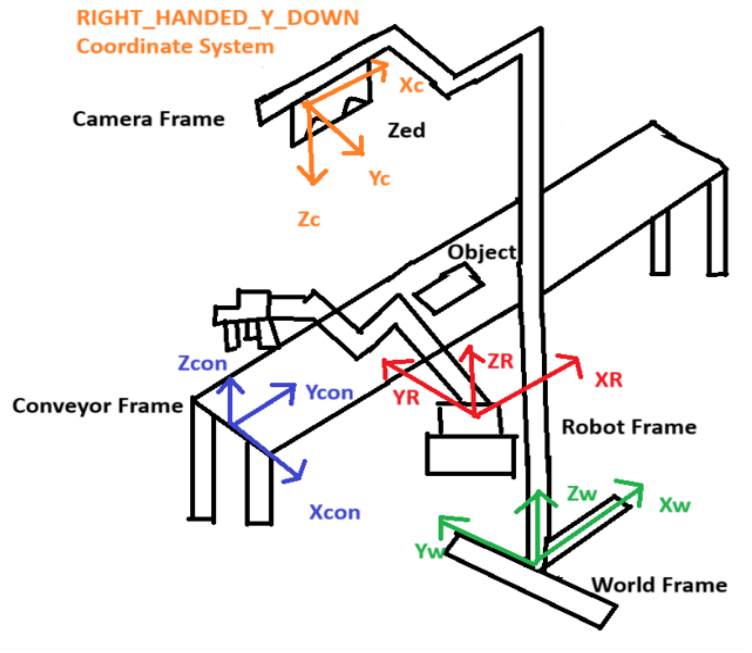


Figure 2.1: Work-cell coordinate system

The robot work cell is structured around four primary coordinate frames, each serving a specific functional role. Figure 2.1 illustrates the structure of the robot work-cell coordinate system. These coordinate frames are identified as follows:

- World Frame (Green)
- Camera Frame (Orange)
- Robot Arm Frame (Red)
- Conveyor Frame (Blue)

Each frame follows a standard three-axis orientation (X, Y, Z), which defines its spatial reference. The intersection point of these axes denotes the origin of each frame, which is critical

for spatial calibration and positional measurements. The placement and orientation of these coordinate frames ensure consistent spatial referencing throughout the robot work cell, enabling accurate positioning, measurement, and coordination of all components.

World Frame:

The origin of the world frame has been manually defined to optimize visibility and facilitate accurate distance measurements between it and other frames. Specifically, the World Frame origin is located at the center of the bottom surface of the stand, serving as the global reference for the work cell.

Camera Frame:

The camera uses the "Right-Handed Y-Down" coordinate system, which is the default configuration for the ZED camera. Users may choose from six different coordinate systems, depending on application requirements, but the default has been maintained in this setup. The origin of the camera frame is located slightly behind the left optical sensor ("eye") of the camera. This frame has been assigned within its own individual system of the camera by default and cannot be changed manually.

Robot Arm Frame:

The robot arm frame is defined by its internal control system and cannot be manually altered. Its origin is located near the first joint of the arm, providing a fixed reference point for kinematic calculations (Ex: Acquire the position coordinates of the end-effector with respect to robot arm frame) and motion control.

Conveyor Frame:

In the same vein as the world frame, the origin of the conveyor frame has been manually defined to optimize visibility and facilitate accurate distance measurements between it and other frames. Specifically, the Conveyor Frame origin is positioned at the center of the left edge of the conveyor belt, aligning with the operational reference point for item tracking on the conveyor.

2.3 Concept of Operations

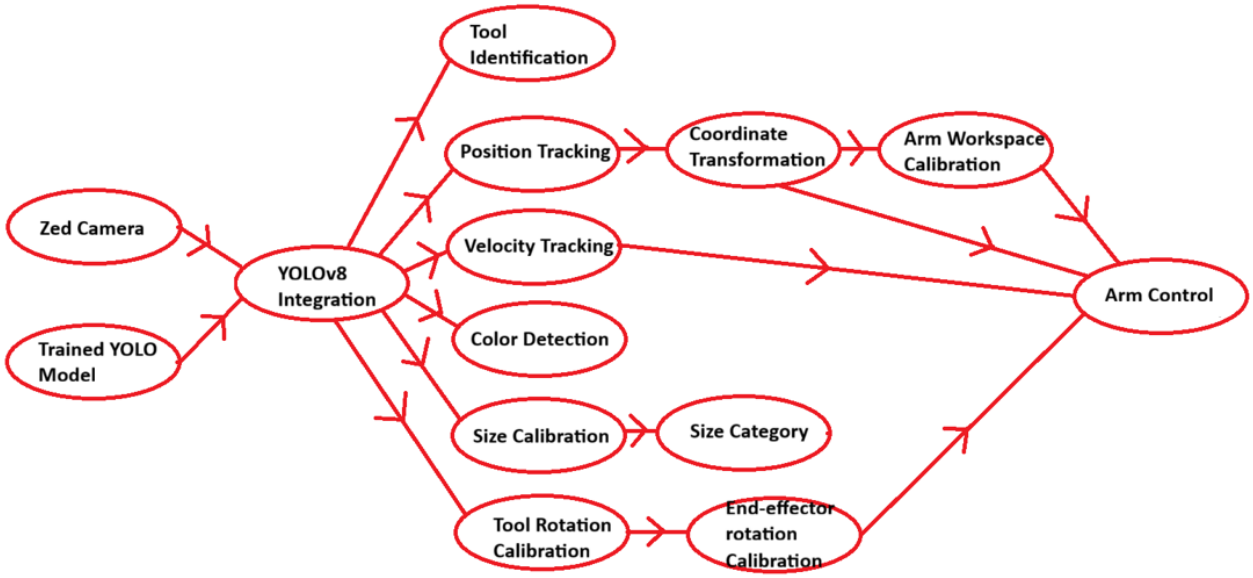


Figure 2.2: Concept of operations of the work-cell

The system as described in Figure 2.2 enables a sophisticated robotic arm to operate in dynamic environments with real-time object tracking, identification, and interaction. With the integration of YOLOv8 model, the robotic arm can perform tasks with high accuracy and efficiency based on the detected features such as tool types, object positions, velocities, and sizes. The main key elements of this diagram can be explained as follows.

- Zed Camera and a trained YOLO Model for a custom trained dataset:

The Zed Camera is used to capture the environment, which is then processed using a trained YOLO model. This model identifies and tracks objects in real-time.

- YOLOv8 Integration:

The central node in the diagram represents the YOLOv8 model integration. This system processes inputs from the camera and identifies relevant features like tool identification, position, velocity, and color.

- Tool Identification:

YOLOv8 is used for detecting tools within the environment. This process is crucial for the system to recognize different tools and objects in the workspace, making it a key component for controlling the arm's interactions.

- Position Tracking:

The system tracks the position of objects and tools within the environment, which is vital for precise arm movements. The position information is then passed on to the Coordinate Transformation stage to convert the detected positions into robotic arm coordinates.

- Velocity Tracking:

This component tracks the movement velocity of the objects, ensuring that the arm’s actions are synchronized with the detected objects' motion. In this project the robotic arm needs to be controlled only if the velocities of the tracking objects are close to zero (The conveyor belt does not move).

- Color Detection:

Specific color detection algorithms are used within an image processing environment. This feature can be useful for identifying the color differences of the detected objects.

- Size Calibration and Category:

Size calibration involves measuring the 3D dimensions of detected tools and objects, while the Size Category node classifies objects based on their size, and identifies whether the object sizes are small, medium, or large.

- Tool Rotation Calibration:

This feature ensures that the rotation of tools and objects is accurately detected and calibrated, allowing for precise manipulation by the robotic arm. High level image processing techniques have been used for this task.

- End-effector Rotation Calibration:

Like tool rotation, this step ensures that the end-effector of the robotic arm (soft gripper) is calibrated for proper orientation and operation to pick up the objects more accurately.

- Arm Workspace Calibration:

This is critical for mapping the space in which the robotic arm operates within the environment of the conveyor belt. This ensures that the arm's movements are restricted within a predefined area to prevent collisions or errors.

- Arm Control:

Finally, all the data from the previous steps (position, velocity, coordinate transformation, and end-effector rotation angle) are used to control the robotic arm. This integration allows for precise, real-time movements and interactions with the tools or objects in the environment.

2.4 Mechanical Diagram:



Figure 2.3: Mechanical structure of the work-cell

Figure 2.3 illustrates the setup of the entire robotic work-cell is designed to automate the process of object handling, where the conveyor transports items, the Zed Camera provides real-time environmental feedback by detecting objects. In this project the objects can be defined as three specific types of tools, and they are wrench, hammer, and screwdriver. The robot arm manipulates those items based on instructions from the computer. In this setup, the Zed Camera likely aids in vision-based object detection, helping the robot arm to identify and interact with items placed on the conveyor (Ex: The location of the object). The task of the conveyor is to move the objects that the robot arm will pick up and place. This supports a steady flow of objects to handle by the robot arm. The computer is running code that likely implements the logic for object detection, robot arm control, and the overall coordination of the system. The software handles tasks like determining the position of objects, calculating movements for the arm, and ensuring synchronization between the components. The robot arm (RotricsDexarm) in this setup can pick up objects from the conveyor, move them to another location. A soft gripper with a pneumatic system is used to pick up and place objects. The arm is likely programed with the improvements of the movements and pneumatics to act more accurately and efficiently.

2.5 Component Block Diagram:

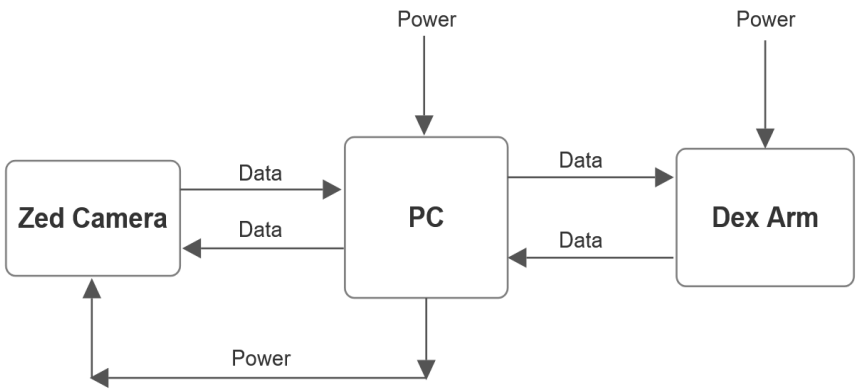


Figure 2.4: Component block diagram of the work-cell

The Figure 2.4 illustrates the interconnection and data flow between three primary hardware components used in the robotic work-cell: the Zed Camera, the PC, and the Dex Arm (Robotic Arm). Each component is powered independently, and data communication flows through the PC. The Zed camera receives power from the PC. It sends real-time image and depth data to the PC and receive control and configuration commands from the PC. The PC acts as the central processing unit of the system. It processes the data from the Zed Camera, interprets the environment, and makes decisions to control the Dex Arm. The PC receives processed visual data from the Zed camera and sends configuration settings to the Zed camera. It also sends control signals and movement instructions to the Dex Arm and gets feedback from it. The PC is powered by an external power outlet. The robotic arm (Dex Arm) is used for manipulation tasks such as moving the end-effector to a specific location, picking and placing objects. It performs actions based on commands from the PC. It also sends feedback to the PC to check with the Zed camera whether the object has been successfully picked up or not.

2.6 High Level Software Flow Chart:

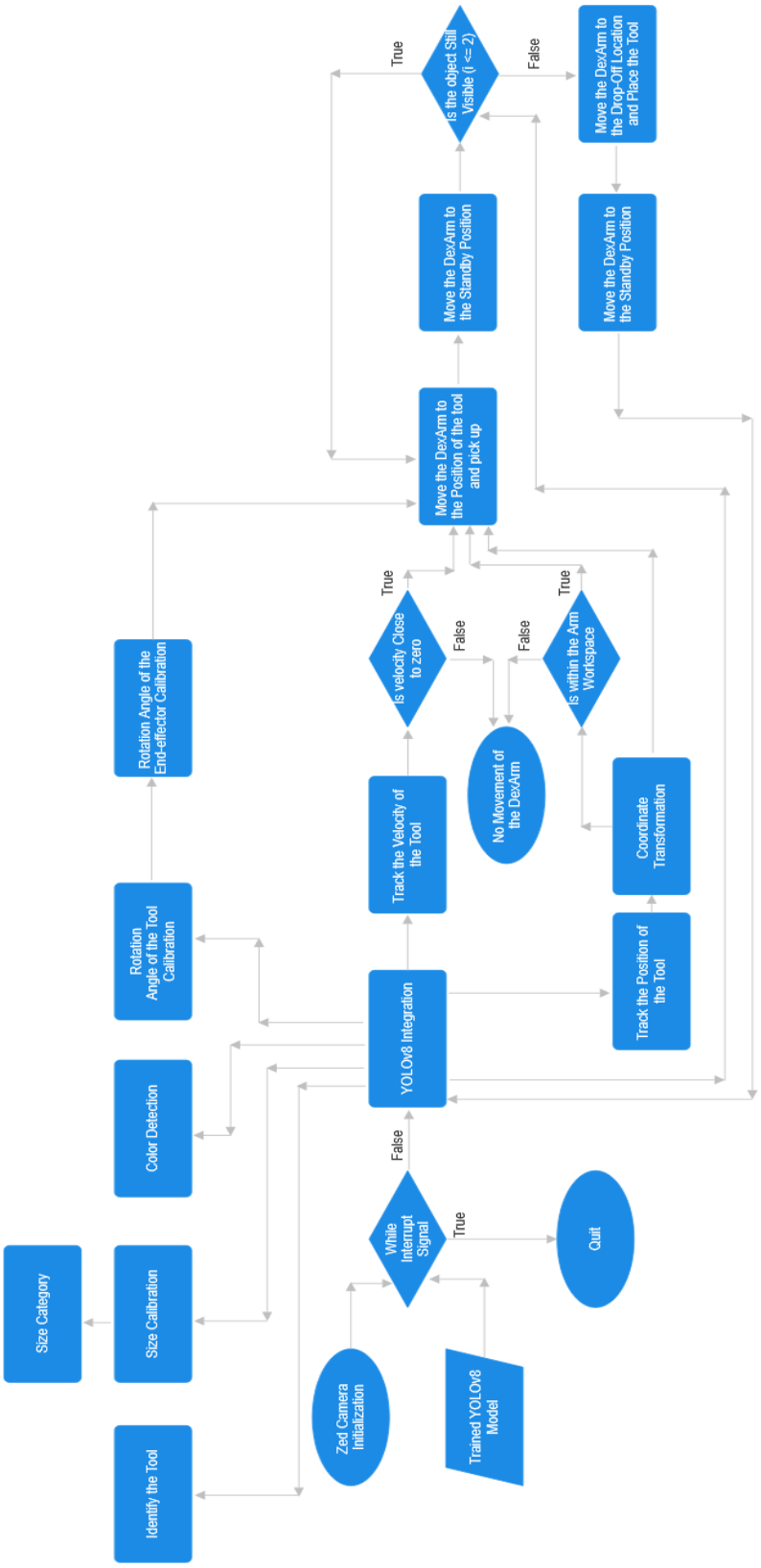


Figure 2.5: High level software flow chart of the work-cell

The flowchart outlines the high-level software design for a robotic system that uses a robotic arm (DexArm), a Zed camera, and a custom trained YOLOv8 object detection model to identify, track, and manipulate detected tools. Figure 2.5 shows how those functions and loops are connected. The system begins by identifying the specific tool (Ex: hammer, wrench, and screwdriver), identifying its color, categorizing its size, and calibrating its size and the rotation angles of it. The Zed camera is initialized, and the YOLOv8 model is used for real-time detection and tracking of the tool. The system continuously tracks the tool's position and velocity, and the orientation (rotation). The movements of the arm are controlled after applying coordinate transformation which represents the position of the tool in robot's frame and calibrating the rotation angle of the end-effector by using the orientation of the tool. Before controlling the DexArm, it is necessary to confirm that the tool's movement is detected to be slow or stationary and the tool is within the workspace area of the robot arm. Otherwise, the arm does not move to the pickup location of the object because of those restrictions. Additionally, the system needs to ensure whether the detected tool remains visible or not during the manipulation of the robot arm. If the arm fails to pick up the detected tool, then it tried to pick up the same tool maximumly two times. Then the arm moves to another tracked location randomly to pick up a different detected tool if it couldn't pick up the tool in two trials. If the pickup successes, then the arm moves to the drop off location to place the tool. If a keyboard interruption occurs, the entire system halts. The overall process involves the integration of computer/camera-based vision, movement and pneumatic controls of the arm for handling detected tools more preciously and efficiently.

Chapter 3

Sub-systems

3.1 Vision System

3.1.1 YOLOv8 for Custom Object Detection

YOLOv8 is the latest iteration of the “You Only Look Once” (YOLO) series of models, setting a new benchmark in the world of computer vision. Released in January 2023, YOLOv8 brings state-of-the-art advancements to tasks like object detection, image classification, and instance segmentation. Developed by the same team behind YOLOv5, YOLOv8 introduces several improvements to both the model architecture and the developer experience. These enhancements include an anchor-free detection approach, refined convolutions, and a streamlined, user-friendly Python package for easier implementation. With a focus on high accuracy and efficiency, YOLOv8 excels on benchmarks like COCO and Roboflow 100, outperforming previous versions and other models in terms of accuracy and generalization. One of the standout features of YOLOv8 is its ease of use, with an intuitive command-line interface (CLI) and a versatile Python API that make it accessible for both new users and experienced developers. The model’s improved architecture enables faster and more accurate object detection, making it a go-to tool for a wide range of computer vision applications. Whether for academic research, industry applications, or edge deployment, YOLOv8 offers an unmatched combination of performance, flexibility, and developer support. Figure 3.1 shows an example of using YOLOv8.

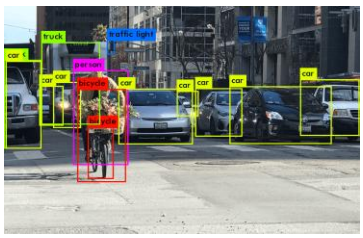


Figure 3.1: Object identification example from YOLOv8 (Used without permission [3])

3.1.2 Integration of the Zed camera with Yolov8 Model

The integration of the ZED camera with the state-of-the-art YOLOv8 (You Only Look Once) object detection model opens the door to powerful real-time 3D object tracking and localization. This process leverages the ZED SDK and TensorRT, which provides high-performance optimization for deep learning models. With the rise of advanced models like YOLOv8, the need for seamless integration between camera systems and AI-driven object detection has never been more critical. The ZED camera, paired with YOLOv8, offers a compelling solution for applications ranging from robotics and autonomous vehicles to augmented reality and surveillance.

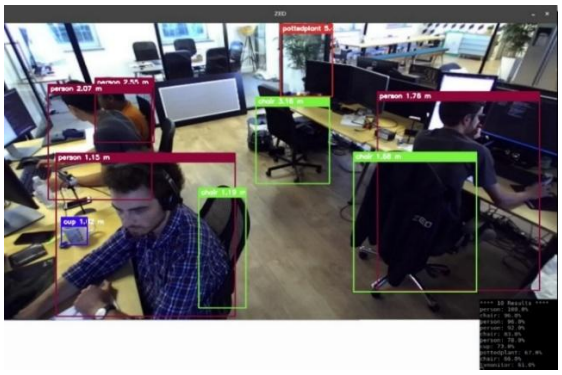


Figure 3.2: Zed camera with YOLOv8 integration (Used without permission [4])

As shown in figure 3.2, in this integration, the ZED SDK is used to capture stereoscopic images, while YOLOv8 (via an ONNX export) detects 2D objects in real-time. Once detected, the ZED SDK adds an extra dimension by extracting 3D information from the camera's depth map, enabling precise 3D bounding boxes, localization, and continuous object tracking. The combined system is optimized for performance through TensorRT, ensuring rapid inference even with complex models. With this integration, the system can visualize a 3D point cloud of detected objects, draw 3D bounding boxes around them, and track them in space with remarkable

precision. Additionally, the ability to use a custom-trained YOLOv8 model gives users full control over the types of objects they wish to detect, expanding the scope of possible applications by providing robust and efficient solutions for object detection and tracking.

3.1.3 Vision Sub-Systems and Their Results

The robot work-cell comprises six key vision sub-systems: tool identification, position tracking, color detection, size calibration, velocity tracking, and tool rotation calibration. A comprehensive overview of each sub-system, along with its respective outcomes, is provided in the following section.

3.1.4 Tool Detection and Position Tracking

This section uses a ZED camera and integrated YOLOv8 module for real-time object detection and 3D tracking of specific tools, such as a Hammer, Wrench, and Screwdriver. The program captures frames from the ZED camera, processes them through YOLOv8 to detect objects, and filters the results based on confidence scores and predefined labels. For each valid detection, the script calculates the 3D position (x, y, z coordinates) of the object using the depth information from the ZED camera and visualizes the detection with a bounding box and 3D coordinates on the frame. Overlapping detections are removed using the Intersection over Union (IoU) metric, and only the most confident detection is retained. The system continuously updates the tracked positions and displays the output in an OpenCV window. The program can be interrupted by sending an interruption signal from the keyboard, at which point the camera is properly closed, and resources are cleaned up. This setup is ideal for applications such as detecting specific objects (tools in this project) and tracking their locations and manipulating a robot arm in 3D space to handle them. The following figure 3.3 shows identifying the type of tools and tracking their positions in 3D coordinates.

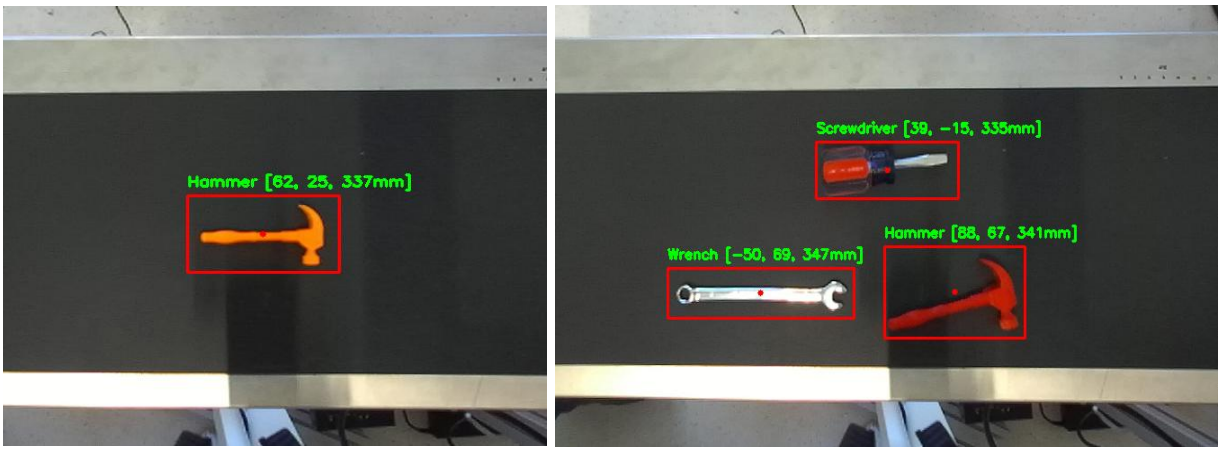


Figure 3.3: Tool identification and tracking their positions

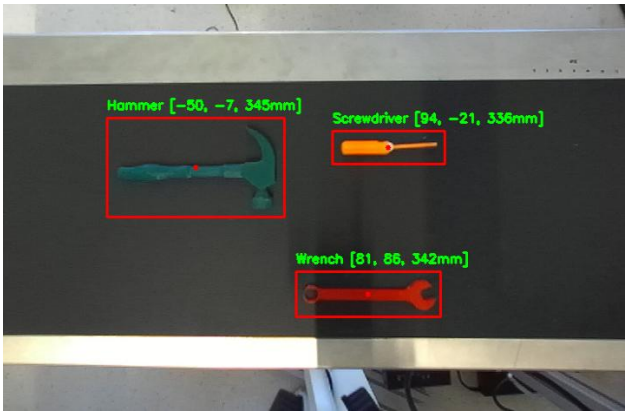
These two figures represent the class identification of the tools and their locations with respect to the camera frame. Single or multiple tools can be detected within a single frame. 2D bounding boxes and the center points of the detected tools are also shown in those figures.

3.1.5 Color Detection

This section detects specific colors in each frame of 2D bounding box of a detected tool by using the HSV (Hue, Saturation, Value) color space. It first converts the RGB (Red, Green, Blue) image to HSV and then creates binary masks for various colors (red, green, blue, orange, yellow, grey, violet, and black) based on predefined HSV color ranges. Morphological operations (opening and closing) are applied to remove noise and fill gaps in the masks. The function counts the number of non-zero pixels in each mask to determine the presence of a color. If the count of non-zero pixels for a color exceeds a threshold (1000 pixels), that color is considered detected. The function then returns the name of the detected color. This method is typically used in computer vision-based image processing that identifies colors of specific objects in an image.

Color detection of tools with different colors:

The following figures illustrate the color identification of three distinct tools. The color of the hammer is identified as "Green," the screwdriver as "Orange," and the wrench as "Red." Those results are shown on a Linux terminal. Figure 3.4 illustrates the color detection of tools.




```
0: 384x640 1 Hammer, 1 Screwdriver, 1 Wrench, 56.6ms
Speed: 3.0ms preprocess, 56.6ms inference, 108.5ms postprocess per image at shape (1, 3, 384, 640)

The information of the detected tool
Type: Hammer
Color: Green
Size(mm): W=121.98, H=67.50, D=11.94
Camera coordinates(mm): X=-50, Y=-6, Z=342
Robot coordinates(mm): X=-85, Y=331, Z=-117

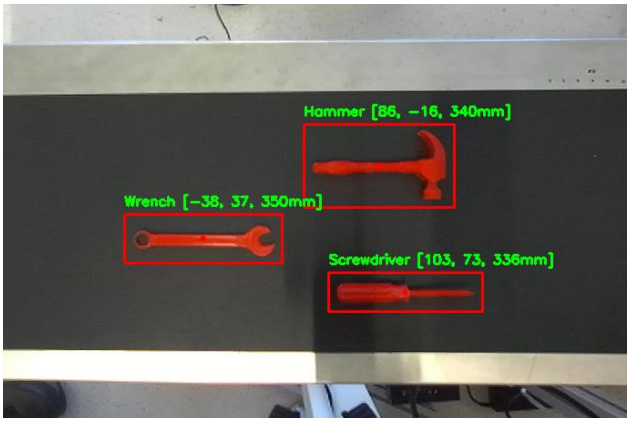
The information of the detected tool
Type: Screwdriver
Color: Orange
Size(mm): W=76.31, H=20.97, D=4.22
Camera coordinates(mm): X=94, Y=-21, Z=337
Robot coordinates(mm): X=59, Y=346, Z=-117

The information of the detected tool
Type: Wrench
Color: Red
Size(mm): W=96.17, H=31.70, D=5.44
Camera coordinates(mm): X=80, Y=86, Z=341
Robot coordinates(mm): X=45, Y=239, Z=-117
```

Figure 3.4: Different color identification

Color detection of tools with the same color:

The following two figures represent the color identification of three distinct tools. All three different tools are identified as ‘Red’. Those results are shown on a Linux terminal. Figure 3.5 illustrates the same color detection of three different types of tools.




```
The information of the detected tool
Type: Wrench
Color: Red
Size(mm): W=101.47, H=32.26, D=8.58
Camera coordinates(mm): X=-38, Y=37, Z=350
Robot coordinates(mm): X=-73, Y=288, Z=-117

0: 384x640 1 Hammer, 2 Screwdrivers, 1 Wrench, 36.5ms
Speed: 3.2ms preprocess, 36.5ms inference, 2.7ms postprocess per image at shape (1, 3, 384, 640)
[Server] Sent to Client 1: {'Type': 'Hammer', 'Color': 'Red', 'Size(mm)': [92.35, 55.34, 11.83], 'Size Category': 'Medium'}

The information of the detected tool
Type: Hammer
Color: Red
Size(mm): W=92.35, H=55.34, D=11.83
Camera coordinates(mm): X=86, Y=-16, Z=340
Robot coordinates(mm): X=51, Y=341, Z=-117
[Server] Sent to Client 1: {'Type': 'Screwdriver', 'Color': 'Red', 'Size(mm)': [98.42, 24.48, 6.91], 'Size Category': 'Medium'}

The information of the detected tool
Type: Screwdriver
Color: Red
Size(mm): W=98.42, H=24.48, D=6.91
Camera coordinates(mm): X=103, Y=73, Z=335
Robot coordinates(mm): X=68, Y=252, Z=-117
[Server] Sent to Client 1: {'Type': 'Wrench', 'Color': 'Red', 'Size(mm)': [101.45, 32.27, 8.66], 'Size Category': 'Medium'}
```

Figure 3.5: Same color identification

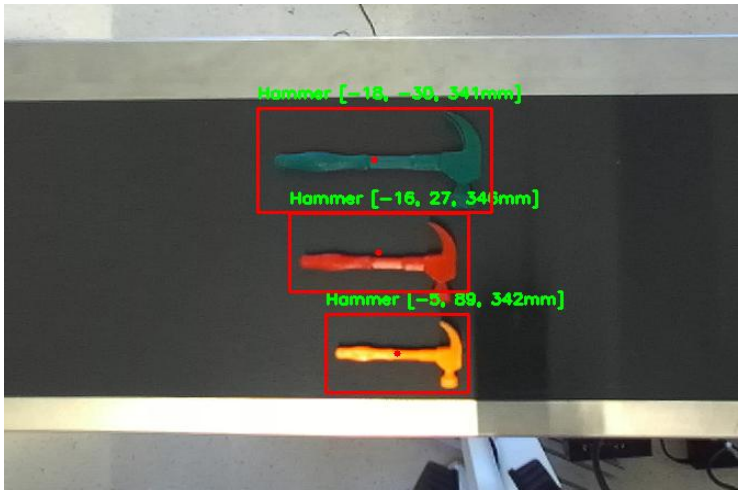
3.1.6 Size Calibration and Identifying the Size Category

This section calibrates the size of an object in a 3D space using data from a ZED camera. It first defines a grid of points by sampling pixel coordinates within a specified region defined by upper and lower boundaries of 2D bounding box with a step size of 3. For each coordinate in this grid, it retrieves the corresponding 3D point from the camera's depth data using a pre-defined function of Zed camera. Valid points are collected in a list after ensuring that no ‘NAN’ values are present. If at least three valid 3D points are obtained, the program filters out outliers by applying

a percentile-based mask to the x, y, and z coordinates of the points. The filtered points are then used to compute the width, height, and depth of the object in the 3D space by calculating the differences between the maximum and minimum values along each axis. If fewer than three valid points are found, the dimensions are set to zero. Finally, the size category that determines whether the tool is small, medium, or large can be distinguished by considering the longest side of the detected tool and applying specific selected threshold values for it. The dimensions of the size are shown in the order of width, height, and depth by using the units' millimeters. It is noticeable that there is a -/+2mm noise when calibrating the size repeatedly in real-time.

Size calibration and category identification of the same tools with different sizes:

The following figures represent identifying hammers with three different sizes. The orange hammer has been classified as 'Small' with dimensions of [75, 45, 11] in millimeters. The red hammer has been classified as 'Medium' with the dimensions of [96, 46, 16] in millimeters. The green hammer has been classified as 'Large' with the dimensions of [125, 58, 16] in millimeters.



```
The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Green', 'Size(mm)': [125.03, 54.33, 16.76], 'Size Category': 'Large'}

The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Orange', 'Size(mm)': [75.71, 45.0, 11.51], 'Size Category': 'Small'}

The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Red', 'Size(mm)': [96.94, 44.66, 16.37], 'Size Category': 'Medium'}

The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Green', 'Size(mm)': [125.16, 57.94, 15.53], 'Size Category': 'Large'}

The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Orange', 'Size(mm)': [75.65, 44.9, 11.25], 'Size Category': 'Small'}

The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Red', 'Size(mm)': [94.79, 44.62, 16.65], 'Size Category': 'Medium'}

The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Green', 'Size(mm)': [125.34, 57.31, 16.17], 'Size Category': 'Large'}

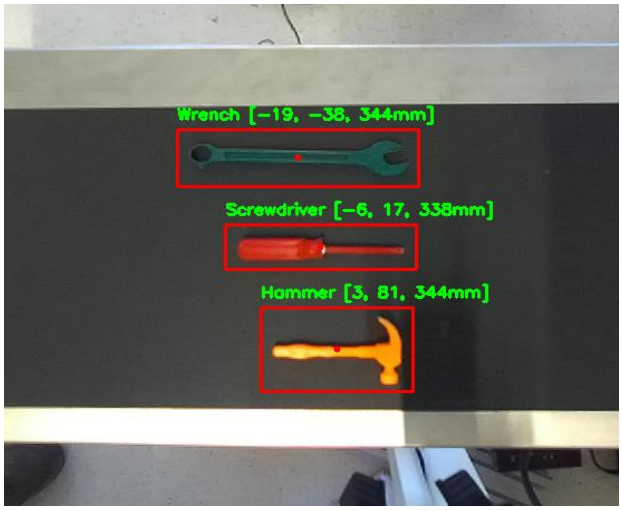
The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Orange', 'Size(mm)': [75.68, 44.87, 11.48], 'Size Category': 'Small'}

The tool details are:
[Client] Received: {'Type': 'Hammer', 'Color': 'Red', 'Size(mm)': [96.8, 42.86, 15.98], 'Size Category': 'Medium'}
```

Figure 3.6: Size calibration and comparison for same type of tool

Size calibration and category identification of three different tools with different sizes:

The following figures represent identifying three different tools with three different sizes. That information is shown in a terminal which is similar to the previous description. Both figures 3.6 and 3.7 represent the size calibration of tools (3D dimensions) in millimeters.



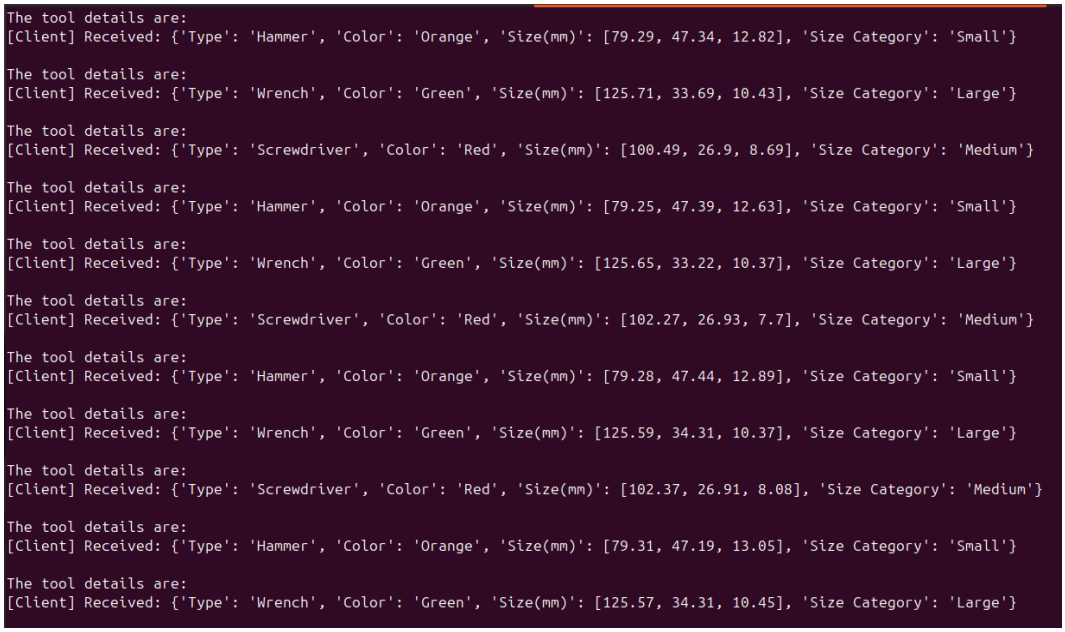


Figure 3.7: Size calibration and comparison for different types of tools

3.1.7 Rotation angles of the tool and arms end-effector calibration

This section calculates the orientation of a region of interest (ROI) in an image using OpenCV. It first converts the ROI to grayscale and applies a Gaussian blur to reduce noise, followed by detecting edges using the Canny edge detector. Then, it finds contours in the edge-detected image. If no contours are found, it returns 'None' and the original ROI. Otherwise, it identifies the largest contour (based on area), calculates the minimum bounding rectangle around it, and extracts the angle of rotation from the rectangle. If the width of the rectangle is smaller than its height, the angle is adjusted by adding 90 degrees to it. The function then draws the rectangle around the contour and overlays the calculated angle as text on the image. Finally, the function returns the computed angle and the annotated ROI. This computer vision image processing program is useful for real-time object detection and orientation tracking, especially when

controlling the DexArm (robot arm) to pick up tools more preciously. Finally, the outcome of this computation (which is the orientation of the tool) is used to calibrate the rotation angle of the end-effector of the DexArm by adding a biased value which is 15 degrees.

3.1.8 Velocity Tracking

This section is designed to calculate and track the velocity of objects detected by the Zed camera in real-time, using positions from previous frames and the current frame. The program takes in current time, positions (the current object positions), and previous positions (the positions and times from previous frames) as input arguments. It first sets a velocity threshold (which is 25 mm/s) to define whether an object's velocity is considered "moving" or "stationary". For each object, if there is previous position data, it calculates the change in positions of 3D coordinates and divides it by the time difference to determine the velocity. If the velocity is below the defined threshold, the object is considered stationary; otherwise, it's moving. If there's no previous data for an object, it assumes the object is stationary. After processing each object, the current position and time are saved for future calculations. Finally, the program returns the velocity status of each detected object, indicating whether it is moving or stationary. If it returns true, the velocity of the object can be considered as close to zero.

Note: Tracking the velocity is not necessarily required to track slow moving objects in real-time because information of the tool updates with a steady frequency.

3.2 Rotrics DexArm (Robot Arm)

3.2.1 Overview

The Rotrics DexArm is a versatile, multi-functional desktop robot arm designed to perform a wide range of tasks such as 3D printing, laser engraving, drawing, and object manipulation. It

allows users to easily swap between different modules like a 3D printer, laser engraver, or soft gripper for diverse applications. Built on open-source firmware and compatible with G-code, it offers flexibility for makers, educators, and DIY enthusiasts. The DexArm can be controlled through Rotrics Studio software, a touchscreen interface, or programming languages like Python and Scratch, making it ideal for educational purposes, prototyping, and creative projects. Whether for precision manufacturing or hands-on STEM learning, the DexArm combines high performance with accessibility, offering endless possibilities for innovation. Figure 3.8 shows the robot arm that has been used for the work cell.



Figure 3.8: Robot arm – 'RotricsDexArm' (Used without permission [14])

3.2.2 Coordinate Transformation

Coordinate transformation plays an essential role in controlling and moving the end-effector of the DexArm (robot arm) to the correct position of the detected tool in the robots' frame. The following figure shows how to convert the 3D position coordinates of the object that calibrated from the Zed cameras' frame to the coordinates with respect to the DexArms' frame. Figure 3.9 is the structure that shows the coordinate transformation between camera and robot arm frames.

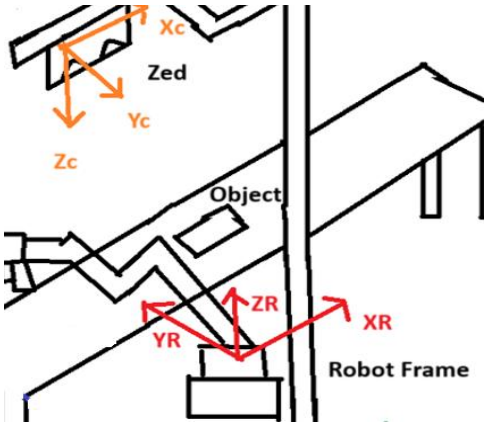


Figure 3.9: Coordinate transformation from the camera frame to the robot arm frame

In the tracking process, the x, y, and z variables represent the distances of the detected object as measured by the Zed camera. The relative distances between the robot frame and the camera frame are fixed, and these values were obtained through manual measurement. Let the following represent the fixed values.

- Xrc = −35 mm
- Yrc = 325 mm
- Zrc = 226 mm

The distances between the DexArm's frame and the object are denoted by Xr, Yr, and Zr. These are calculated based on the following relationships.

- Xr = x – Xrc = X – 35
- Yr = Yrc – y = 325 - y
- Zr = z + Zrc = z + 226 = -117

Note: During real-time data acquisition, some noise in the measurements was observed. As a result, Zr was treated as a constant value of -117.

This constant was determined by first obtaining the depth distance between the robot arm's frame and the surface of the conveyor belt using the "Rotrics Studio" software. To get this constant value, the arm's end effector was moved closer to the surface and observed the depth value that was calibrated by the software. This helps to avoid the risk of the arm going deeper due to noise in the depth measurement, thereby preventing potential damage to the robot.

3.2.3 The workspace of the DexArm

A calibrated workspace for a robot arm is required to avoid collisions and manipulate the arm in an efficient way to accomplish the required tasks. The shaded area of following figure represents the required workspace of the DexArm in 2D space within the robot work-cell. The upper and lower boundaries are represented by those coordinate values. The robot arm only moves if the object is within the workspace. The workspace is shown below by figure 3.10.

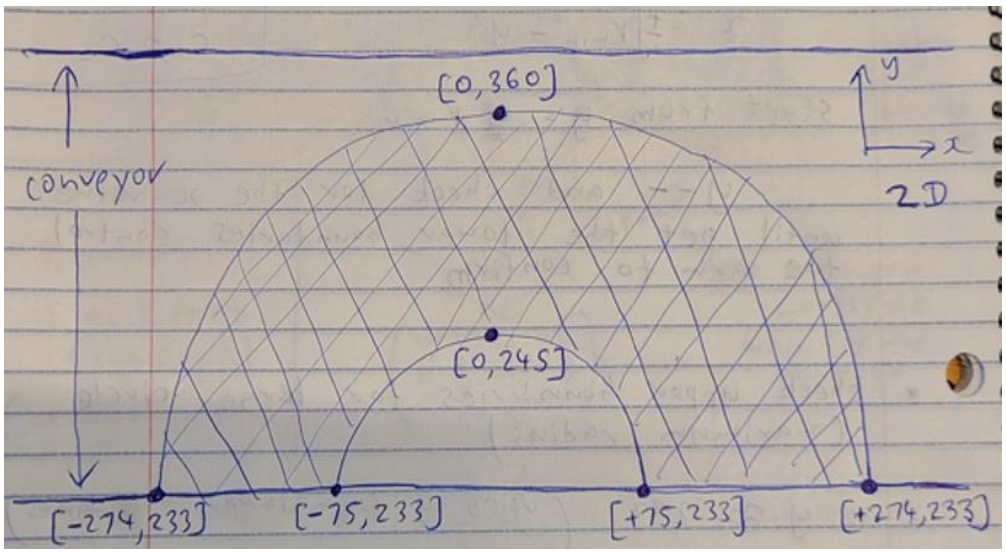


Figure 3.10: Required DexArm workspace

The following figure 3.11 represents the calibrations of both the required workspace area and the workspace volume.

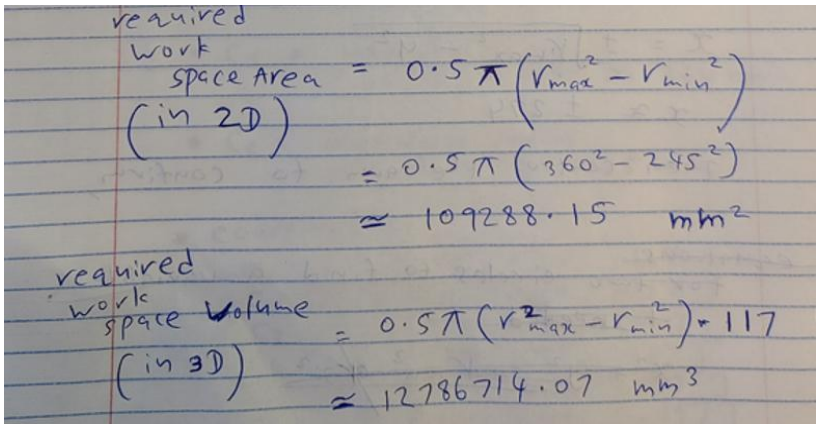


Figure 3.11: Required DexArm workspace area and volume calibration

Check if the object is within the workspace:

The program checks if a 3D point, defined by the robot arms' coordinates to the object, lies within a defined workspace with specific radial and height constraints. The radial distance from the origin in the xy-plane must be between 245 and 360 units, and the height (adjusted with the transformation z = -z + 227) must be between -125 and 135 units. The function calculates the radial distance using the Pythagorean theorem (r = sqrt(x^2 + y^2)) and checks if it falls within the specified range. Similarly, it checks if the transformed z value lies within the allowable height range. If both conditions are met, the function returns True; otherwise, it returns False, indicating whether the object is inside the defined workspace of the DexArm.

3.2.4 Controlling the DexArm using Python API (G-Code) for the Work-Cell

The DexArm is built upon the globally recognized open-source project 'Marlin'. Its firmware is fully open-source, and the control commands (G-code) are fully compatible with 'Marlin'.

There are lots of specific functions used to control the movements of the DexArm and pneumatics for its end-effector. Functions for the RotricsDexarm can be created manually within the class of 'pydexarm' using Marlin G-code and M-code commands. The control of the arm is

done through serial communication, where G-code and M-code commands are sent to the robot. Mainly, the program contents core communication methods, arm movement functions, getting the position and state queries, configuration functions, delay functions, gripper and picket control functions etc. The purpose of these functions of the class ‘DexArm’ is to wrap the complex G-code/M-code commands into user-friendly Python methods, making it easier to control the DexArm for moving, rotating, setting parameters, controlling modules (Ex: soft gripper, air picker) and reading position/state of the arm.

Chapter 4

Final Test Program

The final test program executes all required test commands sequentially, without relying on real-time data transmission. This approach was implemented to verify the correct operation of all specific functions and logic commands in a step-by-step manner. It allows for the early detection of unexpected malfunctions during execution. The program integrates all relevant subsystems, as previously described, and is based on the vision system and the DexArm platform. Figure 4.1 represents the background of the robot work cell with objects (tools).

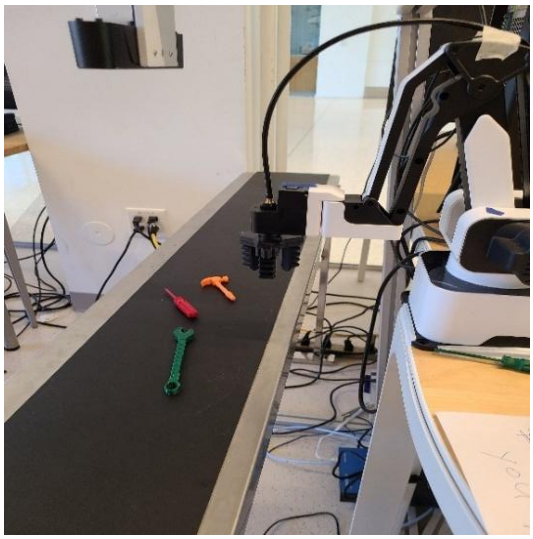


Figure 4.1: Robot work-cell with objects

The program flow summary:

1. Library Initialization and Import Required Files

   Load and initialize all required libraries and dependencies.

2. Helper Function Definitions

   Define utility and support functions used throughout the program.

3. Signal Handling

   Implement interrupt handling to allow the user to terminate the program safely.

4. DexArm Initialization

   Set up and configure the DexArm robotic arm.

5. Load the pretrained ‘YOLOv8’ Model

   Load the YOLOv8 model for the custom trained dataset for identifying specific types of tools (Ex: hammer, wrench, and screwdriver)

6. ZED Camera Initialization and YOLOv8 Integration

   Initialize the ZED camera and integrate YOLOv8 with a custom-trained model for object detection.

7. Tool Detection and Tracking

   Detect tools and track their positions, applying noise filtering techniques for accuracy.

8. Coordinate Transformation

   Convert coordinates from the camera frame to the robot's operational frame.

9. Velocity Tracking

   Monitor the velocities of detected tools to determine stability and readiness for manipulation.

10. Size Calibration and Categorization

    Calibrate tool dimensions and classify them into predefined size categories.

11. Color Detection

    Identify and categorize tools based on their color.

12. Orientation Calibration

    Determine the orientation of detected tools and compute the required end-effector rotation angle.

13. Pick-and-Place Operation

    Control the robotic arm to pick and place tools that are within the arms' workspace and close to stationary.

14. Retry Mechanism

    Reattempt picking a tool if the initial grasp fails, up to a predefined number of retries.

15. Program Termination

    Allow the user to quit the program via an interrupt signal.

The program follows a structured sequence of operations to ensure accurate and efficient tool detection and manipulation of the robot work-cell. It begins with the initialization of all required libraries/files and the definition of helper functions that support the main logic. A signal handler is implemented to allow the user to interrupt and exit the program at any point. The DexArm (robotic arm) is then initialized, followed by the setup of the ZED camera, which is integrated with a custom-trained YOLOv8 model for object detection. Once the system is operational, it detects tools within the camera's field of view and tracks their positions, applying noise-filtering techniques to enhance accuracy. Detected coordinates are then transformed into the robot's operational frame. The program also tracks the velocity of each tool to determine if it is stationary and performs size calibration of the detected tools and then turns into size categories.

In addition, it detects tool colors. Then the program calibrates their orientation to determine the required rotation angle for the robot's end-effector. When a tool is within the robot's workspace and its movement is close to stationary (if below the given threshold value), the arm attempts a pick-and-place operation. If the tool is not successfully grasped, the program reattempts the action up to a defined number of retries. The user can terminate the program at any time through a controlled interruption signal which runs the main loop.

Chapter 5

Real-time Data Transmission

The real-time data transmission is the integrated version of the final test program. This entire program is content with three different subprograms which are used for transmitting the data in real-time. The structure of this program contains one sender and two clients. The Server tracks and send the data of the detected tool for both Client#1 and Client#2. The Client#1 is used to represent the information of the tool that includes the type, color, size, and size category. The Clinet#2 is used to manipulate the robot arm (DexArm). The server also receives feedback from Client#2 for gasp checking and return the arm to the location of the detected tool repeatedly if it would be still visible. The figure 5.1 represents the real-time data transmission diagram which shows that how the sender and two clients are connected.

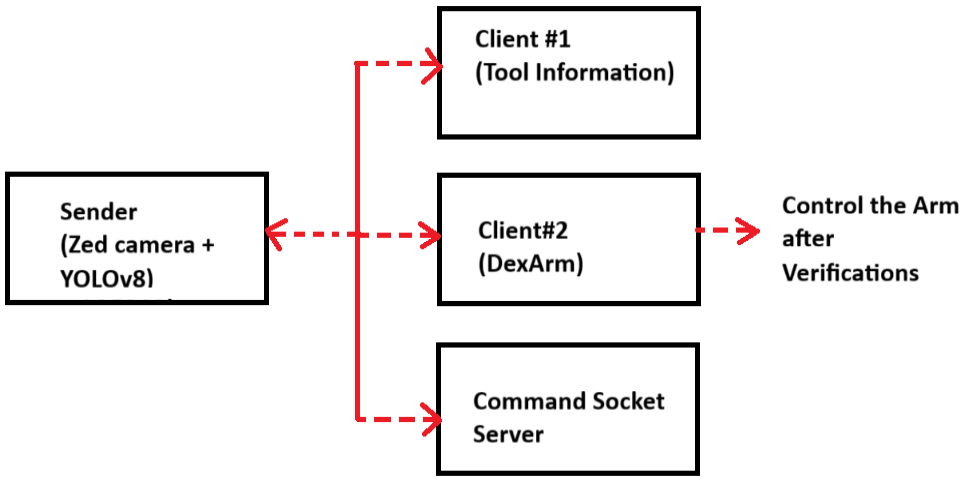


Figure 5.1: Real time data transmission between sender and 2 clients

Sender:

This code integrates a ZED stereo camera with a YOLOv8 object detection model to detect and track tools (hammer, screwdriver, and wrench) in real-time. It captures frames, identifies tools with bounding boxes, estimates 3D positions using depth data, and classifies tool size, color, and orientation. It sends two types of information over socket connections: tool metadata (like size and color) to Client #1 and robot coordinates (like x, y, z coordinates, the end-effector angle, and gasp logic commands) to Client #2. It maintains connections with up to two clients and uses a separate command socket to handle verification requests from Client #2, confirming whether a tool has been picked successfully. The system filters out noise, checks object movement (velocity), and avoids redundant data transmission, ensuring efficient and synchronized tool detection and robot control.

Client#1:

This code connects to the sender via a socket to receive real-time data of the tool, such as the type, size, color, and size category of detected tools. It continuously listens to newline-delimited JSON messages from the sender, decodes them, and prints the tool details to the console. The client ensures proper handling of partial messages using a buffer and includes basic error handling for JSON parsing and connection issues. Its main role is to display the descriptive attributes of the detected tools for monitoring or logging purposes.

Client#2:

This code connects to the sender to receive real-time robot coordinate data for detected tools and uses it to control a DexArm robotic arm for picking and placing the tools. It runs two parallel threads: one listens for incoming commands from the sender, while the other executes those

commands by moving the arm. When a command is received, the client checks if the target position is within the robot's workspace and attempts to grasp the tool. After each attempt, it sends a verification request back to the sender (via a separate command socket) to confirm whether the tool was successfully picked. If verification fails, it retries up to a defined limit. The client also ensures that only one command is processed at a time using a queue and thread-safe flags. The procedure sequence is represented from figure 5.2 to 5.7.

Procedure sequence:

- Stay in the stand-by position

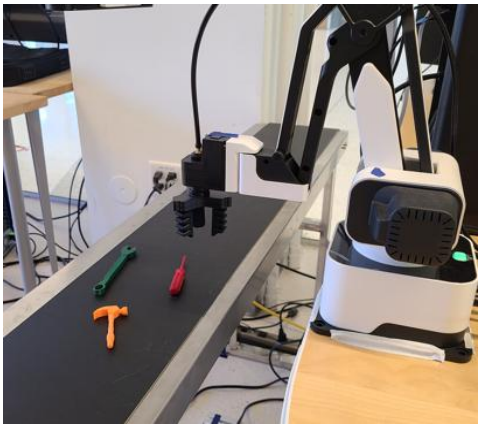


Figure 5.2: Standby position of the arm

- Move to the first tool

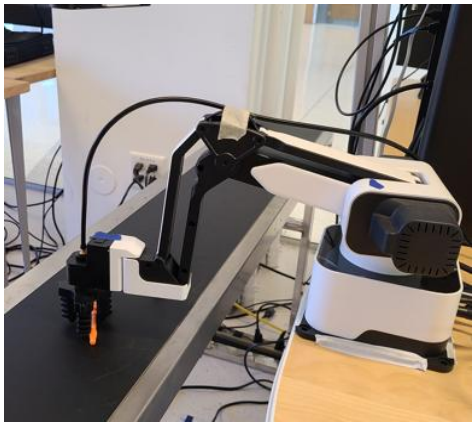


Figure 5.3: Track the coordinates and move

- Pick up the first tool and move to the drop-off location

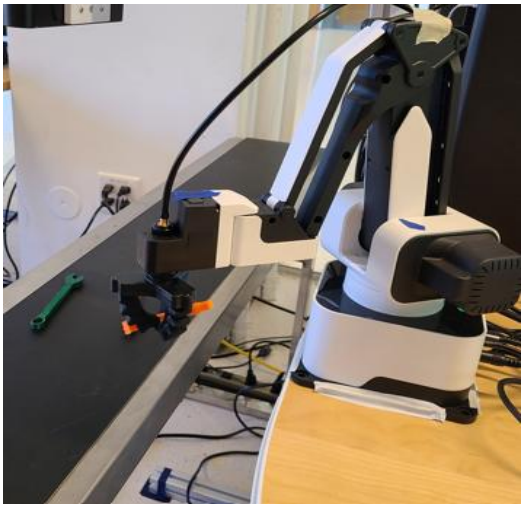


Figure 5.4: Pick up the tool at tool's location

- Place the first tool

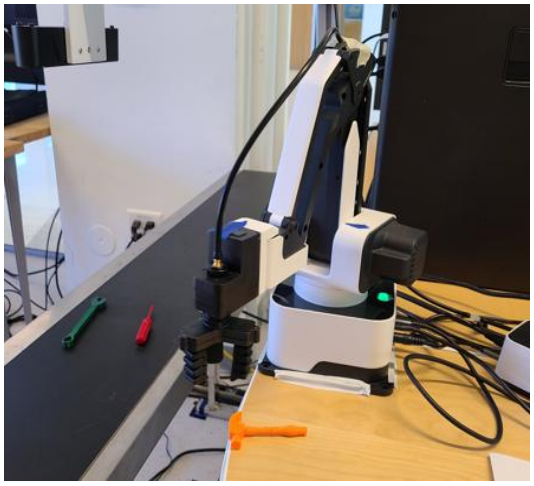


Figure 5.5: Place the tool at the drop-off location

- Move to the second tool and pick it up

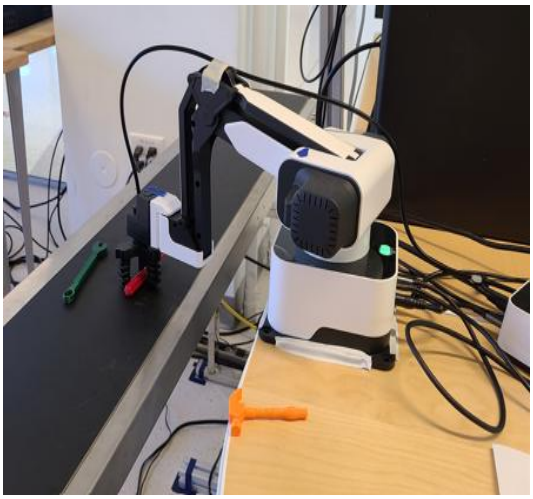


Figure 5.6: Move to the next tool and pick up

- Place the second tool

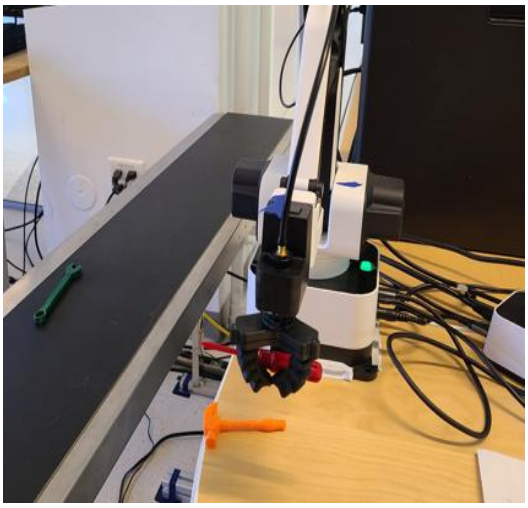


Figure 5.7: Place the tool at the drop-off location

The following sequence represents the control of the arm with **grasp checking** which is used to confirm whether the tool is still visible or not (If the arm misses to pick up the tool). This increases the efficiency and precision of picking and placing tools by DexArm. The procedure sequence is represented from figure 5.8 to 5.13.

Procedure sequence (with grasp logics):

- The arm misses to pick up the hammer



Figure 5.8: Misses to pick up the hammer

- The arm returns to the location of hammer after checking the gasp at the stand-by position and retrying to pick up the hammer again because it is still visible within the defined space of the 3D bounding box (the position changes dx, dy, and dz are less than the defined threshold)

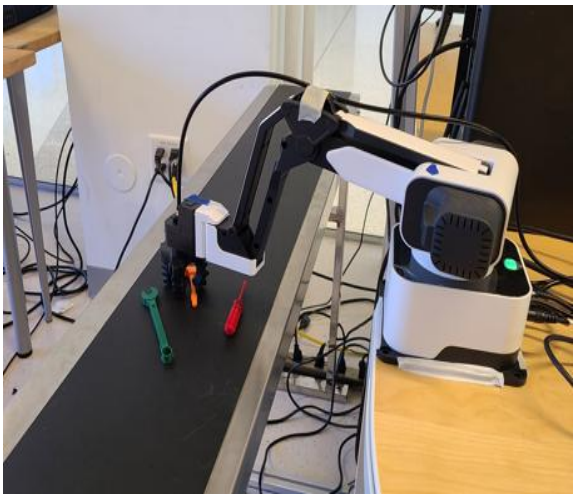


Figure 5.9: Return to the hammer after grasp checking

- Place the hammer at the drop off location after successful pick up



Figure 5.10: Place the hammer at the drop-off location

- Move to the wrench and try to pick it up

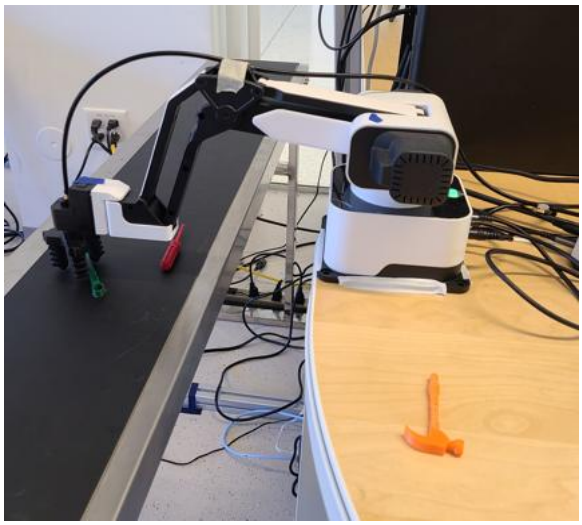


Figure 5.11: Move to the wrench and pick it up

- Place the wrench at the drop off location after successful pick up

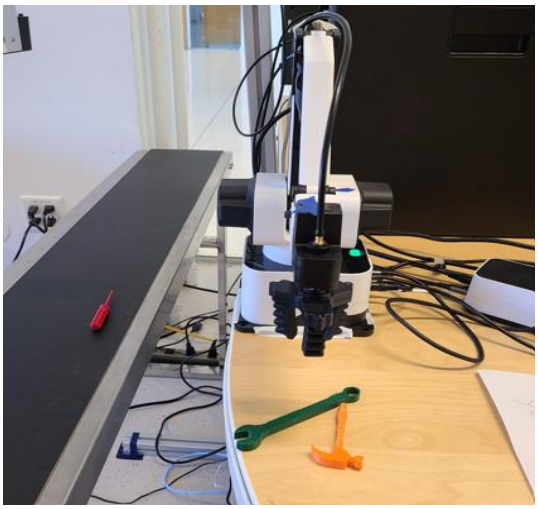


Figure 5.12: Place the wrench at the drop-off location

- Move to the screwdriver and try to pick it up

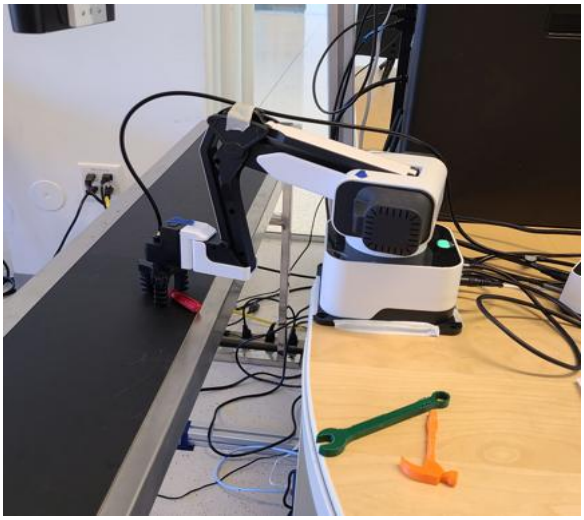


Figure 5.13: Move to the screwdriver and pick it up

The above-described loops of the program continue until all the detected tools have been picked and placed by the DexArm (nly if the tools are within the workspace area of the arm and their velocities are close to zero. Also, those tools should be detectable within the frame of the camera).

Chapter 6

Final Data Acquisition to Calibrate the Accuracy of The Work-Cell

6.1 The Experiment Overview

The following procedures explain how the specific tests were conducted to obtain the results by acquiring data in various conditions. In this experiment, three objects were placed on a conveyor belt, and the DexArm was tasked with picking up all three tools. Each trial required a total of 9 pick attempts to complete the task. The experiment was divided into three test scenarios, each consisting of 10 trials. For each trial, the total number of attempts (which is less than or equal to nine), the number of successful pickups, and the resulting accuracy were recorded. Subsequently, the average accuracy across all trials for each test was calculated at the end of each test. The grasp checking was activated for all the three cases of tests.

6.2 Test Descriptions and Their Results

Test #1: Fixed End-Effector Rotation (No Tool Orientation Consideration)

In this configuration, the end-effector maintained a constant rotation angle which is 15 degrees (biased value for the zero rotation of the end-effector of the arm) throughout the task, and the orientation of the tools on the conveyor was not considered during the pick-up process. Table 6.1 represents the test data and the accuracy of the results of test #1.

Table 6.1: Test#1 Results

Trial #	Total number of tries	Total number of successes	Accuracy
1	9	3	33%
2	8	3	38%
3	9	1	11%

4	9	1	11%
5	9	2	22%
6	9	2	22%
7	6	3	50%
8	9	2	22%
9	7	3	43%
10	6	3	50%

Test#1 Average Accuracy: 30.2%

Test #2: Variable End-Effector Rotation (With Tool Orientation Consideration)

In this test, the end-effector's rotation angle was dynamically adjusted based on the orientation of each tool, allowing for better alignment during pickup. Table 6.2 represents the test data and the accuracy of the results of test #2.

Table 6.2: Test#2 Results

Trial #	Total number of tries	Total number of successes	Accuracy
1	7	3	43%
2	3	3	100%
3	3	3	100%
4	5	3	60%
5	5	3	60%
6	5	3	60%
7	4	3	75%
8	4	3	75%
9	5	3	60%
10	4	3	75%

Test#2 Average Accuracy: 70.8%

Test #3: Variable End-Effector Rotation with Dynamic 3D Position of the Tool and Orientation Update with each grasp checking

In this most advanced configuration, the end-effector not only adapted its rotation angle based on tool orientation but also dynamically updated its 3D position of the tool and orientation after each grasp attempt using real-time feedback. Table 6.3 represents the test data and the accuracy of the results of test #3.

Table 6.3: Test#3 Results

Trial #	Total number of tries	Total number of successes	Accuracy
1	6	3	50%
2	3	3	100%
3	4	3	75%
4	4	3	75%
5	3	3	100%
6	4	3	75%
7	4	3	75%
8	5	3	60%
9	3	3	100%
10	4	3	75%

Test #3 Average Accuracy: 78.5%

6.3 Evaluation of Test Results

The results indicate a clear performance improvement across the three test configurations. Test 3, which represents both variable rotation and real-time updates to the end-effects' position and orientation, yielded the highest average accuracy at 78.5%. This demonstrates the importance of adaptive control strategies in improving object manipulation tasks in robotic systems. There are

several potential future enhancements to the robotic work-cell system that could improve both its accuracy and operational efficiency. Those features have been explained in the next chapter.

Chapter 7

Conclusion

This robot work-cell demonstrates an effective integration of computer vision, real-time communication, and robotic manipulation to automate the detection and handling of tools using a ZED stereo camera and a robot arm (DexArm). The Sender acts as the intelligent central node, combining YOLOv8 object detection with 3D spatial data to identify and characterize tools in real time. The system efficiently distributes responsibilities across two clients: Client #1 monitors detailed tool attributes for analysis or UI display, while Client #2 controls the DexArm to pick and place tools based on precise position and orientation data. The verification mechanism using a separate command socket adds reliability, ensuring accurate tool pickup. Overall, the work cell features a modular, synchronized, and scalable design, rendering it well-suited for applications such as smart manufacturing, automated assembly, and intelligent tool management within industrial settings.

Future Integration Considerations:

To enhance system performance and address current limitations, the following improvements are recommended for future integration:

- Arm Upgrade for Increased Payload Capacity:

  The current robotic arm (DexArm) has a maximum payload capacity of 0.5 kg, which restricts its ability to handle heavier tools. Hence the tools needed to be 3D printed as a requirement for this limitation. It is recommended to consider an alternative robotic arm with a higher payload capacity to enable manipulation of a broader range of objects.

- Shadow Removal in Camera Frames:

  Shadows in the captured frames from the ZED camera are negatively impacting the accuracy of tool identification and color detection. The intensities of the shadows become high during the night-time which affects more for acquiring false results. Implementing a method for shadow removal or compensation would significantly improve the reliability of visual data analysis.

- Enhanced End-Effector Grip:

  The current soft gripper exhibits limitations in securely holding objects. Replacing it with a more robust soft gripper design will improve gripping stability and reduce the risk of object slippage during manipulation.

- Improved Color Detection for Black Objects:

  Due to the predominance of black in the background of the cropped 2D bounding box, color detection for black-colored tools is currently not feasible. Hence, the color detection thresholds for black were needed to be deactivated in the program. To address this, edge detection techniques such as Canny or Sobel can be employed to isolate tool shapes from the background. Subsequently, vision-based image processing algorithms can be applied to accurately identify tool colors, including black.

- Using alternative algorithms for calibrating the orientation of the detected tool:

  There are other algorithms that can be used to calibrate the orientation of the detected tool. More research and test results are required in the future to confirm their accuracy.

References

[1] GAVS sistemi, “What is a Robotic Cell?” [Online]. Available: https://www.gavsistemi.com/what-is-a-robotic-cells/

[2] KUKA, “Powder Measurement Cell,” May 2021. [Online]. Available: https://www.kuka.com/en-de/industries/solutions-database/2021/05/pmc

[3] VisionPlatform.ai, “YOLOv8,” [Online]. Available: https://visionplatform.ai/yolov8/

[4] Stereolabs, “YOLO Object Detection with ZED,” [Online]. Available: https://www.stereolabs.com/docs/yolo/samples

[5] JHFoster, “5 Applications of Robots That Have Revolutionized Manufacturing,” [Online]. Available: https://jhfoster.com/automation-blogs/5-applications-of-robots-that-have-revolutionized-manufacturing/

[6] DIY-Robotics, “Multitasking Robots: Unlocking Efficiency and Adaptability in Industrial Automation,” [Online]. Available: https://diy-robotics.com/multitasking-robots-unlocking-efficiency-and-adaptability-in-industrial-automation

[7] Stereolabs, “Object Detection,” [Online]. Available: https://www.stereolabs.com/docs/object-detection

[8] J. E. Johnson, “What is YOLOv8: Roboflow Blog”, [Online]. Available: https://blog.roboflow.com/what-is-yolov8/

[9] Stereolabs, “ZED SDK for Linux – Installation,” [Online]. Available: https://www.stereolabs.com/docs/development/zed-sdk/linux

[10] Rotrics, “Downloads – Quick Start Guide for the DexArm,” [Online]. Available: https://rotrics.com/pages/downloads

[11] Rotrics, “DexArm Online Manual,” [Online]. Available: https://manual.rotrics.com/

[12] Rotrics, “DexArm Knowledge Base – Software Guide and Troubleshooting,” [Online]. Available: https://rotrics.com/pages/knowledgebase

[13] Marlin Firmware, “G-code Index,” [Online]. Available: https://marlinfw.org/meta/gcode/

[14] Rotrics Manual, “How to re-calibrated DexArm or reset encoder values?”, [Online]. Available: https://manual.rotrics.com/faq-troubleshooting/faqs/how-to-re-calibrated-dexarm-or-reset-encoder-values

[15] Dak A,“ Robot Work Cell Project (Dak Amarasinghe: Capstone Summer 2025)”, Google Shared Drive, [Online]. Available: https://drive.google.com/drive/u/0/folders/0APuxZLxOv2b3Uk9PVA